# WF-A2D: Enhancing Privacy With Asymmetric Adversarial Defense Against Website Fingerprinting

Jianan Huang, *Student Member, IEEE*, Weiwei Liu, *Member, IEEE*, Guangjie Liu, *Member, IEEE*, Bo Gao, *Member, IEEE*, and Fengyuan Nie

***Abstract*—Despite the end-to-end encryption capabilities provided by network protocols such as QUIC in HTTP/3 and the additional tunneling functions offered by proxy tools like virtual private networks (VPNs) and the onion router (Tor), website fingerprinting (WF) techniques can still identify specific network services by exploiting the spatio-temporal characteristics of network traffic. Therefore, defending against WF attacks is crucial for ensuring comprehensive privacy protection for network services. Existing WF defenses typically rely on proxy-based solutions that require coordinated packet manipulations between the client and the proxy node to counteract WF attacks. These symmetric architectures cannot protect network traffic between proxy nodes and web servers from WF attacks. Furthermore, the ability to counter more powerful traffic analysis tools remains a challenging issue. In this paper, we propose WF-A2D, an asymmetric adversarial defense method against website fingerprinting for HTTP/3. WF-A2D employs a two-stage cascading adversarial learning strategy, leveraging packet direction and length patterns to enhance defense performance. Position-based perturbation vectors representing packet operations are generated for packet-by-packet manipulations to achieve real-time WF defense. Experimental results on a real-world HTTP/3-QUIC website browsing traffic dataset demonstrate that WF-A2D can achieve a defense success rate of 97.10% on average against seven state-of-the-art traffic analysis tools, while incurring less than 2% bandwidth overhead. More importantly, WF-A2D can operate independently on the client side and ensure end-to-end protection to web servers.**

***Index Terms*—Privacy protection, website fingerprinting defense, HTTP/3, adversarial learning, defense success rate, bandwidth overhead.**

## I. Introduction

PRIVACY protection has become a pivotal concern for users accessing various web services on the Internet, drawing significant attention from researchers [1]. Encrypted network protocols and tunneling technologies have significantly enhanced the privacy protection of internet users. In recent years, HTTP/3, based on the end-to-end encryption protocol QUIC, has been widely adopted in various network services. This protocol minimizes exposure of information during transmission through encryption certificates, effectively improving the security and privacy of network communications. Additionally, encrypted proxy tools such as VPNs and Tor have gained popularity due to their ability to obscure communication relationships and activities to a certain extent. These tools achieve network service proxying by establishing encrypted tunnels between remote service nodes and clients.

Despite the effectiveness of encryption protocols and tunneling techniques in masking explicit characteristic fields and interaction behavior details of user traffic when accessing network services, a significant amount of spatio-temporal traffic features, such as packet direction and length, can still be exposed [2]. These exposed traffic features can be exploited by website fingerprinting (WF) techniques to identify the network services accessed by users [3]. WF attacks can exploit classical machine learning methods to extract intricate patterns from exposed characteristics of encrypted traffic, or utilize deep learning architectures to perform end-to-end learning directly on encrypted traffic representations. Various sophisticated WF techniques can be deployed in traffic analysis systems at different levels of network gateways. The capacity of these techniques to learn complex traffic features and their ease of deployment pose significant challenges to the privacy protection of network services [4].

Website fingerprinting defense techniques are designed to achieve network service obfuscation by modifying identifiable characteristics of network traffic without disrupting normal communications. These techniques can be broadly categorized into two approaches, both of which utilize additional bandwidth to counter WF attacks [5]. Application-layer defenses, typically implemented through browser plugins or HTTP proxies, modify HTTP requests to obscure traffic patterns [6], [7]. However, these methods operate at a coarse granularity, lacking precise control over individual packet characteristics [8]. In contrast, network-layer defenses enable fine-grained traffic manipulation through packet-level operations such as insertion and padding [9], [10]. Due to their ability to achieve more precise and effective obfuscation, network-layer approaches have become the primary focus of WF defense research [11].

Received 1 August 2024; revised 11 February 2025 and 26 April 2025; accepted 28 April 2025. Date of publication 30 April 2025; date of current version 9 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U2436601, Grant U21B2003, and Grant 61602247. The associate editor coordinating the review of this article and approving it for publication was Prof. Ghassan Karame. *(Corresponding author: Weiwei Liu.)*

Jianan Huang, Weiwei Liu, Bo Gao, and Fengyuan Nie are with the School of Automation, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: jiananwong@njust.edu.cn; lwwnjust@njust.edu.cn; njustgb565@163.com; niefengyuan@njust.edu.cn).

Guangjie Liu is with the School of Electronics and Information Engineering, Nanjing University of Information Science and Technology, Nanjing 210044, China, and also with the Key Laboratory of Intelligent Support Technology for Complex Environments, Ministry of Education, Nanjing 210044, China (e-mail: gjieliu@gmail.com).

Digital Object Identifier 10.1109/TIFS.2025.3565994

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

Despite their advantages, existing network-layer WF defenses face significant challenges. One of the primary limitations stems from the fact that packet-level manipulations often generate traffic that deviates from the expected structure or format recognized by web servers, which generally lack mechanisms for defense-aware processing [12], [13]. As a result, manipulated traffic must be restored before it reaches the web server. To address this issue, most WF defense schemes employ proxy-based architectures, where traffic obfuscation occurs between the client and a remote proxy node [4], [11]. While effective in altering traffic characteristics, these symmetric WF defenses present several practical challenges. First, they require functional modules on both the client and proxy nodes, necessitating tightly coordinated traffic manipulation and restoration. More critically, these defenses do not protect the communication between the proxy node and the target web server, leaving this segment vulnerable to traffic correlation analysis [14]. Furthermore, existing WF defense methods have not adequately addressed the trade-off between bandwidth overhead and defense effectiveness, and their ability to withstand advanced WF attacks remains an unresolved challenge.

Given the challenges of client-proxy coordination and the incomplete privacy protection between the proxy and the web server, proxy-free architectures are more practical. In such architectures, perturbations occur directly between the client and the web server. However, achieving synchronous collaboration between the user and the web server to implement obfuscation is impractical due to deployment effort, coordination overhead, and defense efficiency. Therefore, an asymmetric architecture is more feasible, where the client introduces perturbations into the network traffic, and the web server processes it naturally without modifications [12]. Compared to symmetric architectures, proxy-free asymmetric WF defense architectures offer advantages in terms of end-to-end privacy protection and deployment convenience. Furthermore, to effectively defend against a wide range of WF attacks with minimal bandwidth overhead, it is essential to design packet-wise manipulation strategies based on comprehensive and efficient traffic representations.

In this paper, we introduce WF-A2D, an asymmetric adversarial defense scheme against WF attacks that achieves proxy-free and end-to-end privacy protection for network communication between the user and the target HTTP/3 website. WF-A2D employs a two-stage cascading adversarial learning strategy to optimize the trade-off between defense effectiveness and bandwidth overhead. In the first stage, packet position vectors are generated to determine positions for packet insertion, guided by a discriminator based on packet direction patterns. In the second stage, guided by a discriminator based on packet length patterns, packet operation vectors are generated to refine the position range for packet padding and splitting. Using these pre-generated perturbation vectors, the spatio-temporal characteristics of network traffic can be altered in real time through packet-by-packet operations. The proxy-free defense capability is achieved by leveraging the inherent features of the QUIC protocol, such as padding frames and reliable message delivery, which support an asymmetric defense scheme that does not require packet restoration at the web server.

The main contribution of this study can be outlined as follows:

- We propose an innovative asymmetric adversarial WF defense architecture that enhances end-to-end privacy protection between the client and the HTTP/3 web server. Operating from the client side, it leverages the intrinsic features of QUIC for packet-by-packet manipulations, eliminating the need for packet restoration on the web server. These manipulations are guided by position-based perturbation vectors generated via adversarial learning, providing an efficient and practical architecture for designing WF defense schemes.
- We design a packet manipulation method for real-time WF defense that employs a two-stage cascading adversarial learning strategy. This method is guided by discriminators based on packet spatio-temporal patterns and can be incorporated with various types of discriminators. Using the Insertion Position Generation (IPG) algorithm and the Perturbation Position Range Generation (PPRG) algorithm, packet manipulations can obfuscate spatio-temporal characteristics while achieving a balance between defense performance and bandwidth overhead.
- We utilize three HTTP/3-QUIC datasets encompassing 52,000 samples of website browsing traffic to benchmark the proposed WF defense scheme. The experimental results demonstrate that the proposed scheme significantly outperforms current state-of-the-art WF defenses, achieving an average defense success rate (*DSR*) of approximately 97% against seven advanced traffic analysis tools, with a bandwidth overhead (*BO*) of around 2%.

The remainder of this paper is organized as follows: Section II provides a review of related work in website fingerprinting attacks and defense techniques. Section III compares different WF defense architectures, discusses threat model, and introduces the evaluation metrics used in this study. The proposed WF-A2D is introduced in Section IV. Section V discusses the experimental results. Finally, Section VI concludes this study.

## II. Related Work

This section provides a brief summary of related work on WF attacks and defense techniques. The continually evolving WF technology poses an increasing threat to network privacy, highlighting the urgency of designing easily deployable, efficient, and low-overhead WF defenses.

### A. Website Fingerprinting Attacks

Website fingerprinting attacks are a typical form of network traffic analysis technique. Current traffic analysis predominantly employs either shallow learning or deep learning techniques, relying on fixed protocol field patterns and in-depth traffic characteristics to identify websites. In shallow learning-based approaches, hand-crafted traffic features play a crucial role. In [15], a weight adjustment-based *k*-nearest neighbor (KNN) classifier utilizing over 3,000 traffic features

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

is used for website fingerprinting. Similarly, CUMUL employs the cumulative sequence representation of the first 100 packet lengths, along with support vector machines (SVMs), to identify Tor traffic on a large scale [3]. With carefully selected statistical features, *k*-FP employs random decision forests to balance traffic identification accuracy and computing costs [16]. To further reduce the feature dimensionality, the semi-supervised approach FlowPrint utilizes a limited set of features, including IP addresses, ports, and TLS certificates, to classify mobile application traffic [17]. However, it faces challenges due to the growing use of end-to-end encrypted network protocols and tunneling techniques that obscure certificates and communication relationships. To address this issue, AppSniffer integrates packet length sequences with both machine learning and deep learning, offering an ensemble framework to analyze the intricate patterns present in network traffic [18].

The integration of deep learning represents a significant advancement in website fingerprinting and broader network traffic analysis technologies. In [19], stacked denoising auto-encoders (SDAE) with a three-layer auto-encoder model were constructed to map packet direction sequences into traffic classification results. Subsequently, in [20], packet direction sequences were employed as input features, with a convolutional neural network (CNN) selected as the base model. Following this, a more sophisticated CNN-based model was proposed to achieve higher accuracy [21]. In [22], Var-CNN further improved performance by integrating both packet direction and arrival time within the ResNet-18 architecture. Tik-Tok-WF [23] built upon the CNN-based model proposed by [20], introducing directional timing sequences that integrate timing and directional information to enhance classification performance. Additionally, DeepCorr utilized a CNN-based model to analyze packet direction and arrival time, correlating traffic flows from different connections with related network activities [14]. FS-Net and NeuTic further addressed the limitations in sophisticated classification tasks by utilizing fine-grained representations, particularly packet length sequences [24], [25]. In [26], FlowPic was proposed to convert packet length and arrival time sequences into an image format processed using a modified LeNet-5 model. For identifying more general encrypted traffic, in [27], ET-BERT pre-trained the bidirectional encoder representations from transformers (BERT) model with packet character information and fine-tuned it with task-related network traffic. In [28], a CNN-based model with a traffic aggregation matrix representation, called TAM-CNN, was introduced. This model extracts packet count information from time windows to establish robust traffic fingerprinting.

### *B. Website Fingerprinting Defense Technique*

Along with the development of WF techniques, various defense methods against WF attacks have been explored. These methods can be primarily categorized into experience-driven defenses and adversarial learning-based defenses.

Experience-driven defenses obfuscate network traffic based on observed spatio-temporal characteristics without considering the capabilities of potential adversaries. Among these methods, several methods attempt to achieve traffic obfuscation at the application layer by modifying its behavior. Decoy [6], for instance, functions as a browser plugin that concurrently visits unrelated websites to disguise actual browsing activity. Similarly, HTTPOS [7] randomizes HTTP request orders and injects additional traffic to disrupt distinguishable patterns. However, due to their coarse-grained nature, these defenses struggle to balance bandwidth overhead and obfuscation effectiveness. Furthermore, their reliance on application-layer protocols imposes additional constraints on flexibility. As a result, the majority of WF defense research has shifted toward network-layer approaches, which provide more precise and adaptable protection. For instance, BuFLO sends fixed-length packets at consistent intervals to obfuscate traffic patterns, but it incurs significant bandwidth consumption and latency [29]. To address these issues, Tamaraw adjusts packet lengths and increases upstream sending intervals [30]. Subsequent defense methods introduce more flexibility. In [31], WTF-PAD transmits additional packets during periods of network inactivity. Similarly, in [32], FRONT and GLUE were proposed, which send additional packets at the beginning of traffic flows and between adjacent flows to obfuscate inherent characteristics. In [33], Walkie-Talkie adopts a half-duplex communication method to decrease exposed traffic patterns, though this impractically restricts the inherently full-duplex nature of web browsing. In [34], Glove clusters traffic flows and aligns each flow with a fixed super-trace, but it faces challenges in selecting a super-trace suitable for different traffic flows. Recently, Prism was proposed to increase traffic pattern complexity by modifying packet length distributions towards less common patterns. However, it also requires modifications to the TCP protocol within the system kernel [12].

With WF attacks becoming more sophisticated, adversarial learning-based defenses have been developed to enhance defense performance by simulating potential WF attacks. Generative adversarial networks (GANs) are first used to generate network traffic that is indistinguishable from target traffic. In [35], a GAN with three traffic characteristic dimensions is introduced to mimic other traffic types. In [36], FlowGAN uses six spatio-temporal characteristic dimensions to further enhance obfuscation performance but faces scalability challenges due to the need for distinct GAN models for each website. Recent research focuses on more scalable and effective defense frameworks. BLANKET [9] utilizes randomly generated GAN inputs to modify traffic through a combination of packet insertion and delay. Similarly, Surakav [37] employs a GAN to generate realistic sending patterns by incorporating both packet insertion and delaying strategies. Amoeba [38] introduces reinforcement learning (RL), leveraging classification results as rewards to guide obfuscation through packet padding, splitting, and delaying. Notably, Amoeba's RL agent consistently identifies packet delaying as the least favorable obfuscation action due to the inherent variability of inter-packet timing, which is highly susceptible to network conditions. Consistent with this observation, other studies, such as Mockingbird [39], focus primarily on packet insertion, minimizing the Euclidean distance of packet bursts from a randomly selected target. Dolo [10] pre-generates

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

insertion packet positions with random flipping to enhance unpredictability. To maintain computational efficiency, Mini-patch [40] employs classifier-derived probability labels for targeted perturbations, effectively evading trained classifiers while minimizing overhead.

In summary, the progression of WF defenses has evolved from experience-driven approaches to adversarial learning strategies. These advancements have significantly improved defense performance by enabling more fine-grained and effective packet manipulations. Nevertheless, existing defense designs predominantly depend on intermediary nodes to restore obfuscated traffic before reaching web servers. This dependency arises because the obfuscated packets manipulated by these defense schemes often deviate from the structure or content expected by web servers, leading to incompatibilities when processed directly. While symmetric WF defense architectures, which involve client-server coordination, offer an alternative approach, they often face significant deployment challenges. In particular, endpoint-based symmetric architectures, which require both client-side and server-side modifications, are impractical for large-scale deployment due to the decentralized and heterogeneous nature of web servers. Furthermore, these designs necessitate coordination between the client and the server to determine obfuscation strategies, which increases system complexity and raises compatibility concerns. In contrast, WF-A2D adopts a client-side asymmetric design that leverages the capabilities of QUIC to perform packet-level manipulations without requiring any server-side changes. This design ensures practicality and flexibility for real-world deployment while maintaining high defense effectiveness.

## III. Preliminaries

In this section, we first introduce the proxy-based symmetric WF defense architecture and the proxy-free asymmetric WF defense architecture, highlighting the latter's advantage in providing end-to-end privacy protection without requiring restoration on the target web server. Next, we discuss the threat model for general scenarios, assuming the WF attacker can access the complete traffic between the client and the web server and may utilize any state-of-the-art WF tools to identify the observed traffic. Finally, we introduce the evaluation metrics for WF defenses, including bandwidth overhead, defense success rate, and defense generalization.

### A. Architecture Types in WF Defenses

As illustrated in Figure 1, network-layer WF defense architectures can be broadly categorized into symmetric and asymmetric approaches. In symmetric defenses, proxy nodes are widely adopted in WF research due to their practicality in coordinating traffic manipulation between the client and the proxy node [11]. To ensure normal communication, the original traffic state must be stored at this intermediary before being forwarded to the target web server. In contrast, asymmetric defenses perform traffic manipulation either at the client or the web server, eliminating the need for an intermediary proxy node.

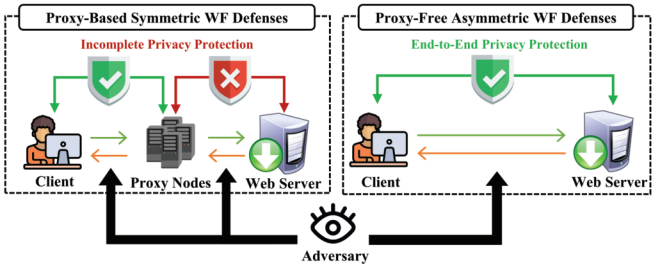



Fig. 1. Two architecture types in WF defenses: proxy-based symmetric WF defenses with incomplete privacy protection, and proxy-free asymmetric WF defenses with end-to-end privacy protection.

Within proxy-based symmetric WF defense architectures, proxy nodes not only relay encapsulated web browsing requests from the user and fetches website resources but also manipulate packets between itself and the client to obfuscate spatio-temporal traffic features. However, this architecture has a notable limitation: it provides incomplete privacy protection. The traffic between the proxy node and the web server remains unprotected, which can be exploited by adversaries to implement WF attacks.

In contrast, proxy-free asymmetric WF defense architectures eliminate the need for a proxy node. This approach ensures end-to-end privacy protection by safeguarding the entire communication path between the client and the target web server against WF attacks. For example, the architecture in this study allows the client to manipulate packets to modify spatio-temporal characteristics without requiring coordinated actions by the web server, making this design more practical and convenient to implement.

### B. Threat Model

WF attacks involve a passive adversary who observes and analyzes traffic flows to identify websites. Understanding the mechanisms of potential WF attacks is crucial for designing and evaluating effective WF defense methods. Without loss of generality, our analysis focuses on the default webpage (e.g., homepage or index page) of a website. This focus aligns with common practices in traffic analysis research, where the default webpage serves as a consistent and representative basis for analysis [41], [42]. Furthermore, we consider an adversary who cannot decrypt or alter the traffic but aims to determine whether the observed traffic originates from commonly accessible websites.

Given the constraints of limited exposed side-channel information and the inherent challenge of concept drift caused by dynamic webpage updates, WF attacks typically analyze bidirectional traffic packets to capture stable traffic patterns and enhance identification reliability [43], [44]. This process can be framed as a supervised classification task. Initially, the adversary transforms the traffic flow into traffic features, which are then analyzed using a classifier to identify specific websites. Formally, the classifier $\mathcal{C}(\cdot)$ is trained on a large dataset $T$, comprising numerous website browsing traffic samples and their corresponding labels $W$. The classifier's parameters $\epsilon^*$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

are optimized to minimize the loss function as described in Eq. (1).

$$\epsilon^* = \arg\min_{\epsilon} \mathbb{E}_{(t,w)\in(T,W)}\{Loss(\mathcal{C}_{\epsilon}(t), w)\} \tag{1}$$

where $t$ denotes a traffic sample associated with label $w$, $\mathbb{E}(\cdot)$ denotes the expectation, and the loss function $Loss(\cdot,\cdot)$ quantifies the discrepancy between the classifier's predictions and the true labels, guiding the optimization of $\epsilon$. Once training is complete, the optimized classifier $\mathcal{C}_{\epsilon^*}(\cdot)$ is employed to implement WF attacks.

In general scenarios, users typically lack knowledge of the adversary's internal workings of WF attack tools. To protect privacy, users must ensure their defense strategies are effective against arbitrary potential WF attacks. Available WF tools should be used to assess and adjust these strategies. Thus, the objective of WF defense is to apply a manipulation configuration $\boldsymbol{v}$ to the web-browsing traffic $t$ using a manipulation $\mathcal{M}(\cdot,\cdot)$, ensuring that the manipulated traffic is misclassified from its original label $w$ by the adversary's classifier.

In adversarial learning-based WF defense framework, a generator $\mathcal{G}(\cdot)$ pre-generates potential vector $\boldsymbol{v}_w$ as the manipulation configuration for the website $w$. A discriminator $\mathcal{D}(\cdot)$ then evaluates its effectiveness within the dataset $T_w$. The perturbation vector $\boldsymbol{v}_w^*$ should maximize the misclassification of the original label by $\mathcal{D}(\cdot)$, ensuring it is also misclassified by the adversary's classifier. The optimization process can be represented by Eq. (2).

$$\boldsymbol{v}_w^* = \arg\max_{\boldsymbol{v}_w} \mathbb{E}_{t_w\in T_w}\{\mathbb{I}[\mathcal{D}(\mathcal{M}(t_w, \boldsymbol{v}_w)) \neq w]\} \tag{2}$$

where $\mathbb{I}[\cdot]$ represents the indicator function that output 1 if its arguments are satisfied and 0 otherwise.

### C. Performance Metrics for WF Defenses

To effectively assess the benefits and costs of WF defenses, it is crucial to consider both the improved defense performance and the increased bandwidth overhead. We utilize the following metrics for a quantitative evaluation:

*1) Bandwidth Overhead (Bo):* *BO* is utilized to quantify the additional data transmission introduced by WF defenses [32]. A lower *BO* indicates a more lightweight defense scheme. It is calculated using Eq. (3) as the ratio of the increase in bytes of the manipulated traffic to the bytes in the original traffic.

$$BO = \frac{\sum_{t\in T}(|\mathcal{M}(t,\boldsymbol{v})| - |t|)}{\sum_{t\in T}|t|} \tag{3}$$

where $|t|$ and $|\mathcal{M}(t,\boldsymbol{v})|$ denote the bytes in the original traffic $t$ and the manipulated traffic $\mathcal{M}(t,\boldsymbol{v})$, respectively.

*2) Defense Success Rate (DSR):* *DSR* is used to evaluate the effectiveness of the defense scheme in evading traffic analysis tools through misclassification [9], which can be calculated by Eq. (4).

$$DSR = \frac{1}{|T|}\sum_{(t,w)\in(T,W)}(\mathbb{I}[\mathcal{C}_{\epsilon^*}(\mathcal{M}(t,\boldsymbol{v})) \neq w]) \tag{4}$$

where $|T|$ denotes the number of traffic samples in dataset $T$. A higher *DSR* indicates a stronger capability to evade traffic analysis.

*3) Defense Generalization (DG):* *DG* evaluates the relative defense generalization of a WF defense scheme against various traffic analysis tools. It is calculated using Eq. (5) as the ratio of the mean defense success rate to its standard deviation across multiple classifiers.

$$DG = \frac{\frac{1}{N}\sum_{i=1}^{N}\Delta DSR(i)}{\sqrt{\frac{1}{N-1}\sum_{i=1}^{N}\left(\Delta DSR(i) - \frac{1}{N}\sum_{j=1}^{N}\Delta DSR(j)\right)^2}} \tag{5}$$

Here, $\Delta DSR(i) = DSR_{def}(i) - DSR_{base}(i)$ denotes the improvement in defense success rate against the $i$-th traffic analysis tool when employing the WF defense scheme. Specifically, $DSR_{def}(i)$ denotes the performance with the defense activated, while $DSR_{base}(i)$ is the baseline performance without the defense. A higher value of *DG* indicates greater stability against multiple WF tools.

## IV. Asymmetric Adversarial Defense Against Website Fingerprinting

This section introduces the proposed WF-A2D scheme. We begin with an overview, followed by a detailed description of the specific adversarial learning components for packet direction and packet length patterns within its two-stage cascading adversarial learning strategy.

### A. Overview

As depicted in Figure 2, WF-A2D is an asymmetric WF defense scheme deployed solely on the client side. It manipulates outgoing packets before QUIC encapsulation and leverages QUIC's inherent features, eliminating the need for additional restoration on the HTTP/3 web server during web browsing. By operating in a proxy-free manner, WF-A2D ensures end-to-end privacy protection between the user and the web server. Three types of packet operations, guided by position-based perturbation vectors generated via adversarial learning, are used for packet manipulations: Packet Insertion (*PI*), Packet Padding (*PP*), and Packet Splitting (*PS*). These operations obscure the spatio-temporal characteristics of website browsing traffic. The pre-generated perturbation vectors are constructed using a two-stage cascading adversarial learning strategy.

*1) Two-Stage Cascading Adversarial Learning Strategy:* This strategy employs two distinct stages to pre-generate the perturbation vector $\boldsymbol{v}_w^*$ for each website $w$, based on previously collected traffic sample set $T_w$. These vectors encode position-based operations for packet insertion, padding, and splitting, specifically designed to obfuscate spatio-temporal traffic patterns. Initially, the strategy obfuscates the packet direction pattern by generating the position vector $\boldsymbol{v}_{w,p}^*$, which determines where dummy packets should be inserted. Prioritizing directional obfuscation ensures that fundamental traffic structures are effectively perturbed, as directional patterns provide critical high-level structural information. WF-A2D then progresses to modifying packet length patterns, generating the operation vector $\boldsymbol{v}_{w,o}^*$, which specifies the perturbation range for packet padding and splitting operations at a finer granularity. These vectors are refined by their respective generators

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

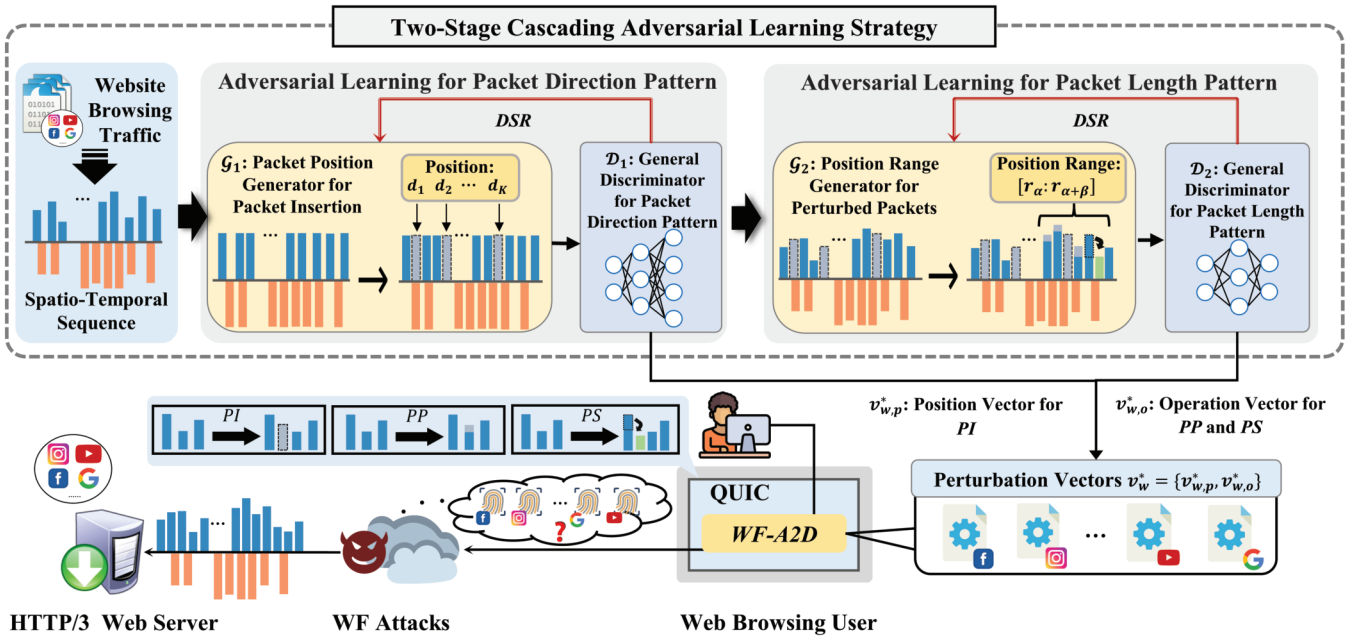



Fig. 2. An overview of the proposed WF-A2D.

through adversarial learning, aiming to obfuscate key features and evade detection by the corresponding discriminators, $\mathcal{D}_1$ and $\mathcal{D}_2$. Notably, these position-based perturbation vectors inherently enable real-time implementation, allowing WF-A2D to track outgoing packet counts and apply pre-determined operations without requiring the full flow trace. This ensures efficient real-time obfuscation with minimal computational overhead. The two-stage cascading adversarial learning strategy can be formally expressed in Eq. (6).

$$\begin{cases} \boldsymbol{v}^*_{w,p} = \arg\max_{\boldsymbol{v}_{w,p}} \mathbb{E}_{t_w \in T_w} \left\{ \mathbb{I}\left[ \mathcal{D}_1\left( \mathcal{M}(t_w, \boldsymbol{v}_{w,p}) \right) \neq w \right] \right\} \\ \boldsymbol{v}^*_{w,o} = \arg\max_{\boldsymbol{v}_{w,o}} \mathbb{E}_{t_w \in T_w} \left\{ \mathbb{I}\left[ \mathcal{D}_2\left( \mathcal{M}(t_w, [\boldsymbol{v}^*_{w,p}, \boldsymbol{v}_{w,o}]) \right) \neq w \right] \right\} \\ \boldsymbol{v}^*_w = \{\boldsymbol{v}^*_{w,p}, \boldsymbol{v}^*_{w,o}\} \end{cases} \tag{6}$$

Specifically, in the first stage, the generator $\mathcal{G}_1$ uses a straightforward representation based on packet direction to generate outgoing packet insertion positions for the *PI* operation. These positions are used as position vector $\boldsymbol{v}^*_{w,p}$ for websites $w$ to obfuscate the packet direction pattern. The packet direction-based discriminator $\mathcal{D}_1$ provides the *DSR* metric to $\mathcal{G}_1$. This feedback enables $\mathcal{G}_1$ to iteratively refine its packet insertion positions until the obfuscated flows successfully deceive $\mathcal{D}_1$. This initial phase lays the foundation for privacy protection by masking fundamental direction pattern of the traffic.

Building upon the initial stage of directional obfuscation, the strategy's second stage shifts its focus to modifying packet length patterns. The generator $\mathcal{G}_2$ utilizes the packet length sequence representation to generate the perturbation range of positions for website $w$. The position range, which specifies the specific *PP* and *PS* operations for packets within it, is used to form the operation vector $\boldsymbol{v}^*_{w,o}$. The packet length-based discriminator $\mathcal{D}_2$ evaluates these modifications and provides *DSR* to $\mathcal{G}_2$. This feedback allows $\mathcal{G}_2$ to iteratively refine the range, balancing defense effectiveness and bandwidth overhead.

*2) Implementation of Packet Manipulations Within QUIC:* WF-A2D adheres to QUIC specifications for packet manipulations and leverages the protocol's features to implement its asymmetric WF defense scheme. A crucial feature for implementing packet manipulations is the padding frame, which is a frame type that carries no semantic meaning and is typically discarded by the receiver upon receipt. This characteristic is particularly useful for implementing *PP* by adding QUIC padding frames to packets, and *PI*, which involves transmitting packets that contain QUIC padding frames. Additionally, QUIC's reliable message delivery mechanism allows upper-layer messages to be split into multiple frames for transmission. The recipient reassembles these frames into the original message, ensuring data integrity before passing it to the upper layers. This capability facilitates the implementation of *PS* without compromising the integrity of the data during transmission. Moreover, the naturally encryption of QUIC not only protects packet contents from external analysis but also plays a crucial role in ensuring the imperceptibility of these packet manipulations.

In contrast, TCP/TLS, another widely used protocol for website browsing, imposes fundamental limitations on packet manipulation. Since TCP/TLS requires all transmitted packets to contain semantically meaningful data, any additional padding bytes must be removed before the web server can correctly process the message. This constraint makes it challenging to manipulate packets without disrupting communication. Additionally, TCP exposes critical metadata, including headers, sequence numbers, and window sizes, which facilitate traffic analysis and increase the detectability of defensive manipulations. This vulnerability is particularly evident in TCP-based defenses such as Cactus [13], which

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

attempts to obscure traffic patterns through retransmissions and dummy acknowledgments. However, these techniques are easily identified and filtered due to the exposed nature of TCP headers and its inherently predictable behavior.

### B. Adversarial Learning for Packet Direction Pattern

In HTTP/3 web browsing, websites typically begin with the loading of essential resources in a request-response mode, leading to initial bursts of packet transmission. These early transmission patterns exhibit distinct spatio-temporal characteristics, which can potentially be exploited to identify accessed network services [45]. In contrast, subsequent traffic can be shaped by dynamic user interactions and asynchronous loading, introducing greater variability and reducing predictability. To mitigate the risks posed by the identifiable initial traffic patterns, we propose the Insertion Position Generation (IPG) algorithm, which obfuscates packet direction patterns through strategic packet insertion during this critical phase.

The IPG algorithm employs adversarial learning to generate a position vector $\boldsymbol{v}^*_{w,p}$ for each website $w$, ensuring that the discriminator $\mathcal{D}_1$ misclassifies samples belonging to $w$ as other websites. To achieve this misclassification, the algorithm iteratively optimizes the positions of insertion packets. It starts by generating a set of $N$ vectors $S_w = \{\boldsymbol{v}^1_w, \boldsymbol{v}^2_w, \cdots, \boldsymbol{v}^N_w\}$ for a given website $w$. Each vector $\boldsymbol{v}^n_w$ in this set is represented as $\{d_1, d_2, \cdots, d_K\}$, where each element indicates a position for one of $K$ insertion packets.

To effectively disrupt the recognizable traffic pattern, the algorithm prioritizes early packet insertion in the traffic flows, distributing these packets according to an exponential decay distribution. This approach ensures that the obfuscation is most pronounced during the critical initial phase of the traffic pattern. To maintain control and effectiveness in packet insertion, the algorithm applies a constraint set $L_i = \{l_1, l_2, \cdots, l_K\}$ in the $i$-th iteration. Each constraint $l_k$ within $L_i$ specifies a range that restricts the allowable insertion position for $d_k$ across all vectors. The constraint set $L_i$ can be formulated by Eq. (7).

$$L_i = \{b_p^{\lfloor \xi_i \cdot e^{-cp} \rfloor} \mid p = 1, 2, \cdots, \mu\} \tag{7}$$

Here, $b_p^{\{k\}}$ indicates that the multiset $L_i$ includes $k$ instances of segment $b_p$. The outgoing packet position space is partitioned into $\mu$ discrete segments, denoted as $\{b_1, \cdots, b_\mu\}$. Each segment $b_p$ spans the range $[(p-1)\cdot\eta, (p-1)\cdot\eta + \lambda]$, where $1 \le p \le \mu$, $\eta$ is the step size between segments, and $\lambda$ is the span of each segment. The decay factor $c$ adjusts the distribution of insertion packets within each segment $b_p$, $\xi_i$ specifies the maximum number of insertion packets per segment and is incremented in each iteration, denoted as $\xi_{i+1} = \xi_i + g$, where the growth factor $g$ dictates the maximum allowable increase in the number of insertion packets per segment $b_p$ for each iteration.

The IPG algorithm then samples each element $d_k$ in every vector $\boldsymbol{v}^n_w$, ensuring it falls within the corresponding constraint $l_k$. It refines and evaluates these vectors, ultimately selecting the position vector $\boldsymbol{v}^*_{w,p}$. During the $i$-th iteration, the discriminator $\mathcal{D}_1$ evaluates each vector $\boldsymbol{v}^n_w$ through $DSR$ to assess its effectiveness in obscuring the correct identification of website $w$. If $\boldsymbol{v}^n_w$ successfully obfuscates all samples from website $w$ such that $\mathcal{D}_1$ classifies each one as belonging to a different website, it is confirmed as $\boldsymbol{v}^*_{w,p}$. If not, the algorithm enters a competition phase to enhance the vector's ability to confuse the discriminator. In this phase, the IPG algorithm generates candidate vectors $\boldsymbol{u}^n_w$ for each $\boldsymbol{v}^n_w$ by randomly selecting three other vectors $\boldsymbol{v}^{r_1}_w$, $\boldsymbol{v}^{r_2}_w$, and $\boldsymbol{v}^{r_3}_w$ within $S_w$. These candidate vectors are constructed to explore potential variations in the vector space and are calculated by Eq. (8).

$$\boldsymbol{u}^n_w = \boldsymbol{v}^{r_1}_w + \psi \cdot \left(\boldsymbol{v}^{r_2}_w - \boldsymbol{v}^{r_3}_w\right) \tag{8}$$

where $\psi$ is a scaling factor ranging from 0 to 1, it is set at 0.5 to balance the exploration capabilities and the convergence speed of the search. Each dimension $d_k$ of $\boldsymbol{u}^n_w$ is checked against its corresponding constraint $l_k$ within $L_i$. If $d_k$ exceeds $l_k$, the value is resampled within the range to ensure compliance.

The discriminator $\mathcal{D}_1$ then evaluate both $\boldsymbol{v}^n_w$ and $\boldsymbol{u}^n_w$ to determine which vector more effectively prevents the discriminator from correctly identifying website $w$. The selection criteria are based on the effectiveness of each vector in deceiving $\mathcal{D}_1$, formulated using Eq. (9).

$$\boldsymbol{v}^n_w = \begin{cases} \boldsymbol{u}^n_w, & \text{if } DSR_1(\boldsymbol{u}^n_w) > DSR_1(\boldsymbol{v}^n_w) \\ \boldsymbol{v}^n_w, & \text{otherwise} \end{cases} \tag{9}$$

Here, $DSR_1(\boldsymbol{x})$ denotes the obfuscation performance of vector $\boldsymbol{x}$ against discriminator $\mathcal{D}_1$. If $\boldsymbol{u}^n_w$ proves more effective at evading $\mathcal{D}_1$ than $\boldsymbol{v}^n_w$, it replaces $\boldsymbol{v}^n_w$. Should $\boldsymbol{v}^n_w$ successfully obfuscate all samples within $T_w$ to evade $\mathcal{D}_1$, i.e., $DSR_1(\boldsymbol{v}^n_w) = 1$, it is then selected as the position vector $\boldsymbol{v}^*_{w,p}$, and the iteration stops.

Considering all vectors in the set $S_w$ still fail to adequately deceive the direction-based discriminator $\mathcal{D}_1$, the IPG algorithm initiates an expansion phase and then moves to the $(i+1)$-th iteration. This expansion is aimed at increasing the variability and diversity of each vector by introducing additional insertion packets, thereby enhancing their ability to confuse the discriminator. In this phase, each vector $\boldsymbol{v}^n_w$ in the $i$-th iteration in the set $S_w$ is adjusted to match the revised constraint set $L_{i+1}$, and is described in Eq. (10).

$$\boldsymbol{v}^n_w = \kappa\left(L_{i+1}, L_i\right) \tag{10}$$

In this expression, $\kappa(\cdot,\cdot)$ functions as a complement operation, and the value of the newly added $(K+k')$-th dimension, denoted as $d_{K+k'}$, is sampled within the constraint $l_{K+k'}$.

Through this iterative process of competition and expansion, the IPG algorithm progresses, continually refining and enlarging the vectors until it either successfully generates a position vector $\boldsymbol{v}^*_{w,p}$ that can obfuscate all samples to evade detection by $\mathcal{D}_1$, or it reaches a maximum number of iterations. This algorithm ensures that each iteration builds upon the previous ones, gradually enhancing the ability to obscure traffic patterns and thereby increasing the defense performance. Considering the simplicity and predictability of fixed insertion packet lengths used in other WF schemes, we formulate a range based on the most common outgoing packet lengths from various websites. The lengths of the inserted packets are then randomly selected from this range. This approach is also applied to other

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

WF schemes involving packet insertion operations to ensure a fair comparison in subsequent experiments.

### C. Adversarial Learning for Packet Length Pattern

The IPG algorithm enhances defense against website traffic analysis tools by obfuscating packet direction patterns. However, by focusing solely on inserting additional packets, IPG fails to fully alter the spatio-temporal relationships between consecutive packets. As a result, the in-deep packet length patterns remain exposed and can be exploited by more powerful traffic analysis tools. To address this issue and enhance defense capabilities, we propose the Perturbation Position Range Generation (PPRG) algorithm, which refines the position range for packet padding and splitting to perturb packet length patterns through adversarial learning.

Perturbing packet lengths in network traffic is analogous to altering pixel color values in images to create adversarial examples [46]. Although this analogy demonstrates the effectiveness of these methods in image processing through gradient descent, their application to network traffic presents substantial challenges. The complexity involved in making precise alterations to each packet, such as deciding whether to pad, split, or modify specific bytes, requires frequent and computationally intensive queries to a discriminator.

Additionally, manipulating the entire traffic flow can result in over-manipulation, complicating the perturbation process and increasing unnecessary bandwidth overhead. Within the two-stage cascading adversarial learning strategy, the IPG algorithm establishes an initial baseline for obfuscation, which helps avoid the complexities and inefficiencies associated with starting from scratch.

Building upon the foundation laid by the IPG algorithm, the PPRG algorithm focuses a specific range to implement packet length patterns perturbations, starting from $\alpha_w^*$ (the initial outgoing packet position for perturbations), and extending to $\alpha_w^* + \beta_w^*$ (the final outgoing packet position for perturbations) to generate the operation vector, denoted as $\boldsymbol{v}_{w,o}^* = \{r_{\alpha_w^*}, \cdots, r_{\alpha_w^*+\beta_w^*}\}$, for each website $w$. Outgoing packets positioned outside this range remain unaltered. Within the range, packet manipulations, including *PP* and *PS*, are predetermined to implement byte-level adjustments.

To generate these manipulations, we employ a piecewise linear function [47]. Its inherent randomness and ability to maintain a uniform invariant density ensure variability in the byte-level adjustments and thorough perturbations of packet length patterns within the range. The function initializes a sequence $\tilde{h}_w = \{h_0, \cdots, h_{\beta_w^*}\}$, with $h_0$ set to provide a context-specific starting point. For simplicity, we use the label number to generate different packet operations among various websites. This sequence is then processed through a transformation function $E_p(\cdot)$, which normalizes each value to create the operation vector $\boldsymbol{v}_{w,o}^*$. Each element in this vector, $r_{\alpha_w^*+i}$ is calculated by Eq. (11).

$$r_{\alpha_w^*+i} = E_p(h_i) = \begin{cases} -\left\lceil \theta_s \cdot \dfrac{h_i}{\varphi} \right\rceil, & \text{if } h_i < \varphi \\ \left\lceil \theta_p \cdot \dfrac{h_i - \varphi}{1-\varphi} \right\rceil, & \text{otherwise} \end{cases} \tag{11}$$

Here, $h_i \in \tilde{h}_w$ and $\varphi$ is set at 0.5 to balance the number of operations between *PP* and *PS*. Parameters $\theta_p$ and $\theta_s$ determine the maximum unit operations for padding and splitting. To ensure that each byte-level adjustment aligns with the typical packet lengths of the website and avoids introducing anomalous packet lengths, a sequence of unique outgoing packet lengths is established. When $r_{\alpha_w^*+i} = k$, the $(\alpha_w^* + i)$-th outgoing packet is extended to match the length of the $k$-th nearest longer packet. Conversely, if $r_{\alpha_w^*+i} = -k$, the $(\alpha_w^* + i)$-th outgoing packet is reduced to match the length of the $k$-th nearest shorter packet, with any remaining payload encapsulated into a new packet. This outgoing packet is then subjected to subsequent manipulations and sent.

Furthermore, the PPRG algorithm introduces a sequential optimization process that iteratively adjusts the perturbation position range $[\alpha_w^*, \alpha_w^* + \beta_w^*]$ based on its impact on defense performance $DSR_2$ through adversarial learning, defending against $\mathcal{D}_2$. The process starts with $\alpha_w^*$ initialized at 0, while $\beta_w^*$ begins at 0 and is gradually increased by a step size $\Delta\beta$ with each iteration. This optimization continues until the improvement in defense performance $\Delta DSR_2^i$ falls below a minimal defense enhancement threshold $\tau$ and is less than the improvement observed in the previous iteration $\Delta DSR_2^{i-1}$. An early stopping mechanism is employed once these conditions are met, limiting the number of additional iterations to $N_{max}$ to prevent overfitting and reduce computational demand.

This ensures that the algorithm identifies the parameter $\beta_w^*$ yielding the best defense performance. Otherwise, the algorithm runs for at most $I$ iterations to find the optimal $\beta_w^*$. After fixing $\beta_w^*$ at its optimal value, $\alpha_w^*$ is optimized in a similar fashion with a step size $\Delta\alpha$. The PPRG algorithm is detailed in Algorithm 1.

## V. Experimental Results and Analysis

In this section, we benchmark the performance of WF-A2D using three publicly available HTTP/3-QUIC datasets under the threat model introduced in Section III-B. To provide comparative and comprehensive results, we compare all the metrics in Section III-C against five typical WF defense schemes using seven state-of-the-art website traffic analysis tools.

### A. Experimental Setup

*1) Dataset:* We employ three HTTP/3-QUIC website browsing traffic datasets [48], collected from the Alexa top-one million ranked websites (as of July 18, 2021) that support QUIC. The data collection process involves downloading website resources via one or more QUIC connections, guided by a pre-generated resource dependency graph that ensures the preservation of dependency and timing relationships among the resources. To verify the continued relevance of these websites, we cross-referenced them with Tranco rankings, confirming that 89 out of 100 monitored websites remain ranked, thereby ensuring the dataset's representativeness. These datasets are divided into training, validation, and testing sets in an 8:1:1 ratio for the subsequent experiments.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1** Perturbation Position Range Generation (PPRG) Algorithm

**Input:** $\Delta\alpha$, $\Delta\beta$: Step sizes for $\alpha_w$ and $\beta_w$;
$\hat{T}_w$: Expected length sequence set for website $w$ after packet insertions;
$\tau$: Threshold for minimal defense performance enhancement between adjacent iterations;
**Output:** $\boldsymbol{v}^*_{w,o}$: Operation vector for website $w$;

**Vector Generation and *DSR* evaluation $\boldsymbol{F_v}(\gamma,\ e)$:**
  $\boldsymbol{v}_{w,o} \leftarrow (\gamma, e)$, potential operation vector generation using Eq. (11);
  $T_w^{opti} \leftarrow (\hat{T}_w, \boldsymbol{v}_{w,o})$, obfuscated length sequence set for website $w$ after applying $\boldsymbol{v}_{w,o}$;
  $DSR_2 \leftarrow (T_w^{opti}, \mathcal{D}_2)$, defense performance evaluation using Eq. (4);
  **return** $DSR_2$;
**Optimize Position Range $\boldsymbol{F_o}(a_0,\ \Delta a,\ b)$:**
  **for** $i \in \{1, \ldots, I\}$ **do**
    $a_i \leftarrow a_{i-1} + \Delta a$;
    $DSR_2^i \leftarrow \boldsymbol{F_v}(a_i,\ b)$;
    $\Delta DSR_2^i \leftarrow DSR_2^i - DSR_2^{i-1}$;
    **if** $\Delta DSR_2^i < \tau$ **and** $\Delta DSR_2^i < \Delta DSR_2^{i-1}$ **then**
      **for** $j \in \{1, \ldots, N_{max}\}$ **do**
        $a_{i+j} \leftarrow a_{i+j-1} + \Delta a$;
        $DSR_2^{i+j} \leftarrow \boldsymbol{F_v}(a_{i+j},\ b)$;
      **end**
      $i^* \leftarrow \underset{i' \in [0, i+N_{max}]}{\arg\max}\ DSR_2^{i'}$;
      **return** $a_{i^*}$;
    **end**
  **end**
  $i^* \leftarrow \underset{i' \in [0, I]}{\arg\max}\ DSR_2^{i'}$;
  **return** $a_{i^*}$;
$\alpha_w^*, \beta_w^* \leftarrow 0, 0$;
$\beta_w^* \leftarrow \boldsymbol{F_o}(\beta_w^*,\ \Delta\beta,\ \alpha_w^*)$;
$\alpha_w^* \leftarrow \boldsymbol{F_o}(\alpha_w^*,\ \Delta\alpha,\ \beta_w^*)$;
$\boldsymbol{v}^*_{w,o} \leftarrow (\alpha_w^*, \beta_w^*)$, operation vector generation using Eq. (11);
**return** $\boldsymbol{v}^*_{w,o}$

- The full-page dataset FPD: This dataset utilizes the Chromium browser to download web resources, reflecting real online activities more closely. It includes 20,000 samples from 100 monitored websites and 10,000 unmonitored websites.
- The single-connection dataset SCD: This dataset employs the QUIC client library to download web resources over a single connection. It contains 20,000 samples, distributed among 100 monitored websites, each contributing 100 samples, and 10,000 unmonitored websites.
- The multi-connection dataset MCD: Using the QUIC client library, this dataset downloads web resources over multiple connections. It consists of 12,000 samples, with 100 monitored websites each providing 100 samples, alongside 2,000 unmonitored websites.

*2) Baselines for Performance Evaluation:* We compare WF defense schemes against seven state-of-the-art website traffic analysis tools. FS-Net [24] utilizes an auto-encoder with a bidirectional GRU to analyze packet length sequences. NeuTic [25] captures long-range dependencies and dynamics of packet sequences through self-attention mechanism and multi-kernel convolution. DeepCorr [14] employs a CNN-based model to establish spatio-temporal correlations among traffic samples from the same websites, leveraging sequences of packet time and direction. AppSniffer [18] combines a light gradient boosting machine with FastAI models in a stacked ensemble, focusing on packet length sequences for traffic classification. TAM-CNN [28] employs a CNN with an aggregation matrix to extract features from time windows associated with packet counts. Var-CNN [22] uses a dual ResNet-18 model with dilated causal convolutions to analyze packet direction and time information. It further incorporates manually extracted statistical features, enhancing the model's ability to perform traffic analysis. Tik-Tok-WF [23] utilizes the same CNN architecture as DF [21] but adopts the directional timing representation, combining packet timestamps with directional information. Additionally, the burst-level timing features proposed in the study are also incorporated to further enhance the identification performance. To ensure a comprehensive evaluation, seven WF schemes are assessed using the FPD dataset, all of which demonstrate strong capability in identifying undefended traffic, achieving $F1$ *score* of at least 90%. Among them, TAM-CNN, Var-CNN, and Tik-Tok-WF integrate timing features with other types of information, such as packet direction and packet count. However, when these schemes are restricted to using only timing features, their performance degrades significantly, with $F1$ *score* dropping to the range of 50%-60%. This result indicates that timing features alone are insufficient for accurate classification. Based on these findings, we retained the original traffic representations used by these WF schemes in our experiments to ensure a fair and consistent evaluation of defense effectiveness.

We introduce five WF defense schemes for performance comparison: WTF-PAD [31] utilizes pre-configured distributions to conceal timing gaps by inserting packets. FRONT [32] inserts packets on the initial segments with the Rayleigh distribution to obscure identifiable features. Mockingbird [39] minimizes the Euclidean distance between the packet direction bursts of obfuscated and randomly selected traffic through adversarial learning. Minipatch [40] inserts blocks of packets tailored for different websites through adversarial learning, aiming to minimize bandwidth overhead while defending against WF attacks. Prism [12] disrupts spatio-temporal traffic characteristics on packet length with a transition matrix through packet padding and splitting.

*3) Experiment Configuration:* For traffic analysis tools, each classifier is configured with the hyperparameters recommended in its original publication to achieve optimal training and classification performance in the neural network models. Moreover, controlling the amount of traffic information accessible to each classifier is critical. We regulate this by adjusting the sequence length of each flow. Although longer sequence lengths provide more insights, they increase computational overhead and do not always yield additional valuable information. To determine the optimal sequence lengths, we

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

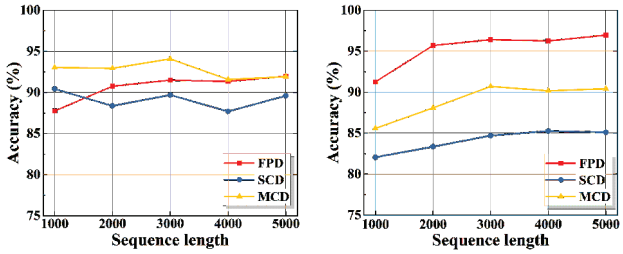



Fig. 3. Accuracy of different sequence lengths is evaluated using (a) AppSniffer; (b) TAM-CNN.

use AppSniffer and TAM-CNN as benchmarks. We vary the sequence lengths and evaluate classifier's performance using accuracy metric.

The results, depicted in Figure 3, indicate that performance for both classifiers improve as the sequence length increases from 1,000 to 3,000. Beyond this point, the performance gains plateau, and AppSniffer experiences a decline, likely due to the inclusion of redundant data that does not provide additional benefits. Based on these findings, we standardize the sequence length at 3,000 for subsequent experiments. We truncate longer sequences and pad shorter ones to ensure consistency across traffic data inputs.

To ensure a fair comparison between WF-A2D and other WF defense schemes, we adhere to the parameters recommended in the respective original papers for each defense scheme. We configure FRONT with $N_s = N_c = 500$, $W_{min} = 0.2s$, and $W_{max} = 3s$. WTF-PAD employs the *normal_rcv* distribution. Mockingbird operates under Case II with DF as the detector. Minipatch also uses DF as its detector, employing adaptive tuning to refine perturbations. Prism is set in an untargeted defense mode with client-side perturbations. Following established research [12], [40], the adversarial learning-based WF defense schemes utilize testing data to generate perturbations and validate defense performance with non-overlapping validation data, and other schemes directly obfuscate the validation data. All codes are written in Python and executed on a computer with an Intel i9-14900KF CPU, 64 GB RAM, and an NVIDIA GeForce RTX 4090 GPU with 24 GB memory.

### B. The Influence of Parameters

We analyze three main parameters in adversarial learning that influence the defense performance of WF-A2D: the discriminator $\mathcal{D}_1$, the initial outgoing packet position for perturbations $\alpha^*$, and the number of perturbed packets $\beta^*$. Additionally, other parameters have been explored and determined on a limited number of flow samples. In the adversarial learning in packet direction pattern, we set the vector number $N = 20$, the number of segments $\mu = 30$, the step size between segments $\eta = 15$, the span of each segment $\lambda = 30$, the decay factor $c = 0.125$, the maximum number of insertion packets $\xi_1 = 5$, and the growth factor $g = 2$. In the adversarial learning for packet length pattern, we set the step size $\Delta\alpha = \Delta\beta = 5$, the maximum unit operation $\theta_p = \theta_s = 5$, the additional iteration

TABLE I

DEFENSE PERFORMANCE DSR AND BANDWIDTH OVERHEAD *BO* OF DIFFERENT $\mathcal{D}_1$ IN THE ADVERSARIAL LEARNING FOR PACKET DIRECTION PATTERN

| | FPD | | SCD | | MCD | |
|---|---|---|---|---|---|---|
| | *BO* (%) | *DSR* (%) | *BO* (%) | *DSR* (%) | *BO* (%) | *DSR* (%) |
| DF | 1.36 | 96.81 | 3.20 | 95.74 | 3.85 | 85.70 |
| AWF | 1.26 | 94.15 | 3.15 | 95.30 | 3.66 | 86.62 |
| Var-CNN-dir | 1.24 | 87.43 | 3.04 | 93.71 | 3.43 | 71.52 |

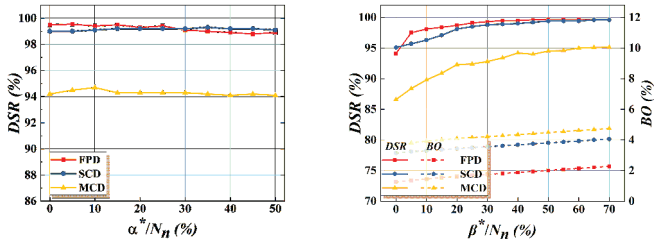



Fig. 4. (a) Defense performance *DSR* for different $\alpha^*/N_n$; (b) Defense performance *DSR* and bandwidth overhead *BO* for different $\beta^*/N_n$ are evaluated in the adversarial learning for packet length pattern, respectively.

$N_{max} = 5$, the threshold $\tau = 0.02$. We select $k$-FP [16], which includes packet length patterns in its traffic features, as the discriminator $\mathcal{D}_2$.

In the adversarial learning for packet direction pattern, we evaluate three well-known direction-based website traffic analysis tools for $\mathcal{D}_1$: AWF [20], DF [21] and Var-CNN [22]. To ensure fairness, only the packet direction component of Var-CNN is utilized, denoted as Var-CNN-dir. Additionally, the parameters $\alpha^*$ and $\beta^*$ are fixed at 0, and we utilize the discriminator $\mathcal{D}_2$ to evaluate the *DSR*.

Table I demonstrates that using AWF as the discriminator $\mathcal{D}_1$ in adversarial learning for packet direction pattern results in better defense performance with reasonable bandwidth overhead, outperforming other tools in *DSR*. In contrast, DF achieves similar *DSR* but incurs higher bandwidth costs generally. On the other hand, Var-CNN-dir, which exclusively utilizes the direction component, shows lower performance with the least bandwidth overhead. Across all three datasets, we observe a positive correlation between increased *BO* and enhanced *DSR*. However, employing a more effective discriminator allows for the early discontinuation of packet insertions that do not contribute significantly to defense performance. Given its optimal balance of high defense performance and reduced bandwidth overhead, AWF is chosen as the discriminator $\mathcal{D}_1$ for adversarial learning in packet direction pattern.

For the adversarial learning of packet length pattern, we analyze the impact of $\alpha^*$ and $\beta^*$ on defense performance. Since the total number of outgoing packets varies for each website, we normalize $\alpha^*$ and $\beta^*$ by dividing them by the total number of outgoing packets $N_n$ in each traffic sample. This normalization uses $\alpha^*/N_n$ and $\beta^*/N_n$ to reflect perturbation additions across different websites. Figure 4(a) shows the change in *DSR* as $\alpha^*/N_n$ increases from 0 to 50%, with $\beta^*/N_n$ fixed at 40%. In the MCD dataset, perturbations starting at the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON OF WF-A2D AND FIVE WF DEFENSE SCHEMES AGAINST SEVEN ADVANCED TRAFFIC ANALYSIS TOOLS (MEAN±STD). BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST PERFORMANCE, RESPECTIVELY

| | | *BO*(%) | *DSR* (%) | | | | | | | | *DG* |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | FS-Net | NeuTic | DeepCorr | AppSniffer | TAM-CNN | Var-CNN | Tik-Tok-WF | Aver | |
| FPD | Undefended | - | 1.41±0.0344 | 8.65±0.1645 | 13.29±0.1847 | 8.30±0.0535 | 3.60±0.0781 | 1.71±0.0638 | 3.12±0.0595 | 5.73±0.0912 | - |
| | Random | 1.81 | 15.95±0.0981 | 28.76±0.2759 | 46.32±0.3875 | 16.28±0.1441 | 8.58±0.0624 | 8.40±0.1623 | 13.45±0.0828 | 19.68±0.1733 | 1.41 |
| | FRONT [32] | 5.93 | 59.49±0.1639 | 77.32±0.1752 | 83.75±0.2495 | 77.46±0.1388 | 62.74±0.2063 | 81.11±0.1316 | 73.38±0.1729 | 73.61±0.1769 | 9.30 |
| | WTF-PAD [31] | 2.32 | 52.84±0.1629 | 65.85±0.2380 | 83.65±0.2718 | 70.45±0.1670 | 21.53±0.1851 | 78.18±0.1922 | 27.28±0.1882 | 57.11±0.2007 | 2.29 |
| | Mockingbird [39] | 6.49 | 28.87±0.2097 | 83.12±0.1815 | 83.75±0.2437 | 88.91±0.1042 | 57.55±0.1997 | 79.70±0.1513 | 74.66±0.1872 | 70.94±0.1825 | 3.48 |
| | Minipatch [40] | 0.75 | 10.29±0.1852 | 45.44±0.3305 | 55.31±0.3674 | 61.54±0.3174 | 38.21±0.3195 | 57.17±0.3581 | 51.21±0.1976 | 45.60±0.2965 | 2.53 |
| | Prism [12] | 6.97 | 95.01±0.1204 | 85.30±0.2352 | 83.54±0.2588 | 80.65±0.2332 | 73.92±0.3262 | 88.38±0.2527 | 81.14±0.3130 | 83.99±0.2485 | 8.84 |
| | WF-A2D | 1.81 | **99.39**±0.0357 | **98.17**±0.0748 | **99.48**±0.0348 | **99.98**±0.0099 | **83.75**±0.2352 | **99.60**±0.0398 | **99.32**±0.0430 | **97.10**±0.0676 | **13.73** |
| SCD | Undefended | - | 4.13±0.1084 | 19.90±0.2202 | 12.79±0.1265 | 10.30±0.0844 | 15.33±0.2025 | 6.18±0.1199 | 8.63±0.1208 | 11.04±0.1404 | - |
| | Random | 3.25 | 19.31±0.2558 | 24.68±0.3085 | 33.68±0.2857 | 15.14±0.2787 | 24.41±0.2459 | 14.80±0.3466 | 18.37±0.2059 | 21.48±0.2753 | 1.81 |
| | FRONT [32] | 13.42 | 80.93±0.1750 | 88.97±0.1580 | 85.84±0.2284 | 85.82±0.1169 | **84.43**±0.1887 | 89.49±0.1232 | 83.74±0.1553 | 85.60±0.1636 | **15.17** |
| | WTF-PAD [31] | 5.16 | 90.17±0.1187 | 90.93±0.1734 | 92.81±0.1845 | 93.85±0.0844 | 71.81±0.2685 | 94.44±0.0899 | 82.21±0.2173 | 88.03±0.1624 | 6.98 |
| | Mockingbird [39] | 7.53 | 64.37±0.2687 | 89.26±0.1537 | 82.92±0.2645 | 86.93±0.1367 | 73.75±0.1834 | 92.22±0.1066 | 82.78±0.1943 | 81.75±0.1868 | 7.42 |
| | Minipatch [40] | 0.45 | 37.64±0.2786 | 66.13±0.3171 | 65.62±0.2938 | 68.68±0.2662 | 59.24±0.3095 | 74.95±0.2744 | 70.87±0.3294 | 63.30±0.2956 | 4.35 |
| | Prism [12] | 3.06 | 87.76±0.1684 | 85.28±0.2419 | 87.50±0.2043 | 88.97±0.1296 | 69.46±0.3101 | 92.83±0.1267 | 76.93±0.2603 | 84.10±0.2060 | 6.45 |
| | WF-A2D | 3.25 | **98.45**±0.1076 | **96.35**±0.1846 | **96.67**±0.1034 | **98.76**±0.0640 | 81.08±0.2746 | **99.90**±0.0099 | **99.12**±0.0512 | **95.76**±0.1136 | 8.14 |
| MCD | Undefended | - | 3.22±0.0727 | 14.60±0.1900 | 5.47±0.1085 | 5.92±0.1517 | 9.31±0.1395 | 4.96±0.0943 | 3.96±0.1476 | 6.78±0.1292 | - |
| | Random | 4.51 | 11.41±0.1685 | 37.62±0.3175 | 47.42±0.3725 | 21.46±0.2404 | 43.33±0.3092 | 19.70±0.3028 | 12.59±0.2662 | 27.65±0.2824 | 1.62 |
| | FRONT [32] | 13.91 | 62.58±0.2311 | 76.43±0.2326 | 83.23±0.1947 | 76.33±0.1728 | 77.27±0.1871 | 79.29±0.1835 | 77.41±0.1863 | 76.08±0.1983 | 10.30 |
| | WTF-PAD [31] | 4.70 | 72.76±0.2190 | 76.67±0.2635 | 90.94±0.1631 | 86.10±0.1512 | 69.13±0.2538 | 78.48±0.2415 | 69.92±0.2394 | 77.71±0.2188 | 7.53 |
| | Mockingbird [39] | 6.54 | 42.83±0.2642 | 77.85±0.2184 | 75.21±0.2142 | 69.91±0.2005 | 74.81±0.1972 | 81.52±0.1415 | 79.55±0.1881 | 71.67±0.2034 | 5.25 |
| | Minipatch [40] | 0.76 | 18.20±0.2353 | 50.05±0.3536 | 58.44±0.4244 | 50.03±0.4260 | 73.16±0.3261 | 52.22±0.3802 | 69.78±0.3590 | 53.13±0.3578 | 2.65 |
| | Prism [12] | 4.02 | 80.27±0.2817 | 82.83±0.2589 | 96.35±0.1485 | 93.00±0.1473 | 91.93±0.1793 | 95.45±0.1081 | 89.21±0.2314 | 89.86±0.1936 | 10.26 |
| | WF-A2D | 4.51 | **95.08**±0.1558 | **93.33**±0.1828 | **99.90**±0.0184 | **97.90**±0.0856 | **92.30**±0.2240 | **98.99**±0.0700 | **96.41**±0.1658 | **96.27**±0.1289 | **14.66** |

10% percentile are preferred for most websites. In contrast, other datasets do not show this phenomenon, with different websites having their own preferences for the $\alpha^*$.

Figure 4(b) illustrates the relationship among $\beta^*/N_n$, *BO*, and *DSR* across three datasets, with $\alpha^*/N_n$ fixed at 0. As $\beta^*/N_n$ increases from 0 to 70%, both *BO* and *DSR* exhibit an upward trend. However, *DSR* experiences a rapid increase initially, followed by a plateauing effect at $\beta^*/N_n = 40\%$. Beyond this point, further increases in $\beta^*/N_n$ lead to higher overhead without yielding significant improvements in *DSR*. This observation suggests that setting $\beta^*/N_n$ to 40% represents an optimal trade-off between defense performance and bandwidth overhead.

### C. Comparison With Other Defenses Schemes

We compare WF-A2D with five WF defense schemes across seven state-of-the-art traffic analysis tools, utilizing bandwidth overhead *BO*, defense success rate *DSR*, and defense generalization *DG* as evaluation metrics to assess the effectiveness of each scheme. To provide a baseline for comparison, we also introduce a purely random obfuscation strategy. This random strategy applies packet insertion, padding, and splitting at randomly selected positions sequentially, with the total bandwidth overhead matched to that of WF-A2D. This design allows us to compare the performance of WF-A2D against a non-optimized approach, and also serves as a baseline for evaluating other defense schemes. As shown in Table II, the random strategy provides an average *DSR* improvement of only 15%, with significantly lower *DG* compared to other defense schemes. These results highlight the limitations of random manipulations as an obfuscation method and validate the necessity of optimization-based approaches for achieving robust and generalizable performance.

WF-A2D exhibits the strongest overall performance, outperforming other schemes in both defense success rate and generalization while maintaining reasonable bandwidth overhead. This advantage stems from its use of adversarial learning to comprehensively obfuscate spatio-temporal features. By integrating a discriminator that incorporates timing characteristics, WF-A2D effectively disrupts spatio-temporal patterns without introducing additional latency. As a low-latency defense scheme, WF-A2D proves highly effective against classifiers such as Var-CNN and Tik-Tok-WF, which rely on timing information combined with directional sequences for identification. Although its effectiveness is slightly reduced against TAM-CNN, the latter's performance remains inherently constrained by the variability of timing features. On the MCD dataset, QUIC's support for multiple connections and concurrent streams further increases traffic complexity, introducing diverse timing fluctuations that undermine TAM-CNN's reliability in accurate identification.

Prism, recognized as the state-of-the-art WF defense scheme, surpasses all others except for WF-A2D in terms of defense performance. Both Prism and WF-A2D are asymmetric WF defense schemes, yet they differ significantly in several aspects. First, in the browser-driven FPD dataset, Prism exhibits a notably higher *BO*. This substantial increase in *BO* raises concerns regarding its efficiency in practical web browsing scenarios. Additionally, Prism needs to modify TCP within system kernel to implement its obfuscation, which adds complexity. Furthermore, Prism is less adaptable to evolving web technologies. Its obfuscation strategy relies heavily on TCP's streaming transmission characteristics, which become less effective with the increasing adoption of HTTP/3 built on UDP. Moreover, Prism's reliance on TCP restricts its generalizability. When using encrypted proxy tools to obscure communication relationships further, Prism only obfuscates

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III

DEFENSE PERFORMANCE UNDER ONE-PAGE SCENARIO (MEAN±STD). BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST PERFORMANCE, RESPECTIVELY

| | *Precision* (%) | *F1 score* (%) | *DSR* (%) |
|---|---|---|---|
| Undefended | 98.26±0.0496 | 95.24±0.0831 | 6.92±0.1172 |
| FRONT [32] | 64.54±0.4538 | 24.76±0.2148 | 83.51±0.1590 |
| WTF-PAD [31] | 51.47±0.4555 | 18.97±0.2153 | 86.80±0.1832 |
| Mockingbird [39] | 76.20±0.3844 | 31.55±0.2072 | 78.71±0.1579 |
| Minipatch [40] | 75.01±0.3733 | 58.03±0.3604 | 47.95±0.3626 |
| Prism [12] | 36.71±0.4408 | 16.04±0.2173 | 87.82±0.1993 |
| WF-A2D | **15.06**±0.3294 | **7.90**±0.1932 | **93.98**±0.1593 |

the TCP connection between the client and the proxy node. It cannot modify the additional TCP connection between the remote proxy node and the target web server, resulting in incomplete privacy protection. In contrast, WF-A2D employs the two-stage cascading adversarial learning strategy to optimize the trade-off between defense performance and bandwidth overhead. It also implements packet manipulation within the QUIC protocol, offering greater flexibility and adaptability, which enhances privacy protection for HTTP/3 website browsing.

In the realm of other WF defense schemes, FRONT stands out by introducing significant bandwidth overhead to evade various traffic analysis tools, while achieving enhanced performance in *DG*. Although it slightly outperforms WF-A2D in resisting TAM-CNN and achieves the highest *DG* within the SCD dataset, this advantage comes at the cost of nearly quadrupling *BO* for a marginal 4% increase in *DSR*. Mockingbird occasionally outperforms Prism in evading some traffic analysis tools, but its higher *BO* diminishes its attractiveness. A significant limitation of Mockingbird is its dependency on complete website traffic information, which restricts its capability in processing real-time traffic since it cannot foresee future packet flows. Minipatch tries to minimize bandwidth overhead through a lightweight, adaptive tuning method. However, its straightforward packet insertion strategy, coupled with a less effective adversarial learning component, makes it vulnerable to various traffic analysis tools. WTF-PAD, regarded as a classical WF defense scheme, now struggles to maintain its efficacy against advanced traffic analysis tools. Although it introduces an adaptive mechanism to obscure communication behavior during idle periods, outperforming Prism in the SCD dataset, it does not effectively obfuscate traffic pattern during resource transmissions. Overall, these schemes, despite offering diverse defensive approaches, generally fall short in terms of convenience and effectiveness when compared to the WF-A2D.

Following the recommendation in [49], we conduct further evaluations of WF defense schemes under a one-page scenario. This scenario involves training a binary classifier where each monitored website is treated as the positive in turn, with all other traffic classified as negative. This approach allows the classifier to concentrate on the specific patterns of the monitored website, thereby enhancing its analytical capabilities. For our experiments, we utilize the MCD dataset, which has a relatively small number of unmonitored websites, facilitating faster model convergence and making it more challenging for WF defense schemes to effectively defend against traffic analysis. We employ AutoGluon [50], an automated machine learning framework, to train classifiers and conduct evaluation. AutoGluon's capabilities for automated model selection, ensembling, and hyperparameter tuning are particularly well-suited for this complex and extensive experiment. Furthermore, we assess the effectiveness of our approach using traditional classification metrics, including *Precision* and *F1 score*, as well as the *DSR*, as presented in Table III.

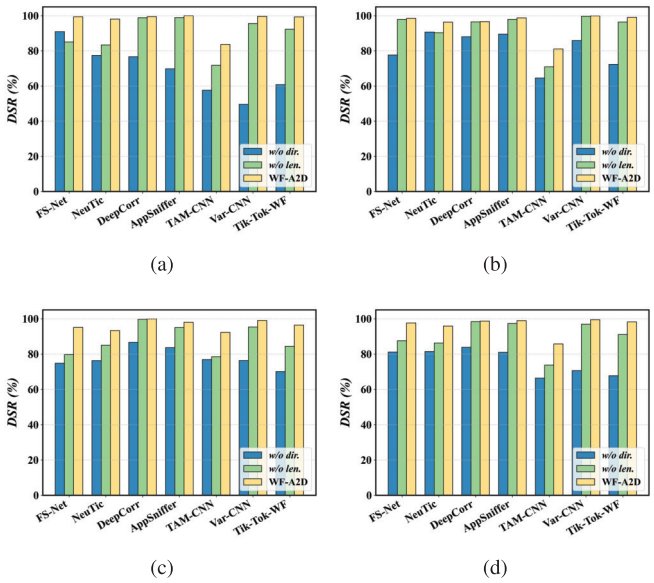



Fig. 5. Ablation analysis of each component evaluated in the (a) FPD dataset, (b) SCD dataset, (c) MCD dataset, and (d) Average performance across three datasets.

In the evaluations, the WF-A2D and Prism defense schemes maintain their effectiveness across all metrics in the one-page scenario. Although *DSR* has decreased compared to the previous scenario, both defense schemes still demonstrate improvements over the undefended case, with an increase in *DSR* of at least 80%. Notably, WF-A2D achieves an increase of 87.06%. It is worth to notice the substantial variations in the *Precision* metric among all defense schemes, a key factor since higher *Precision* indicates greater performance for an adversary in identifying accessed websites. While schemes such as Mockingbird, Minipatch, FRONT, and WTF-PAD show only modest reductions in effectiveness, WF-A2D stands out by decreasing this metric by more than 80%, thereby demonstrating its exceptional ability to protect user privacy even in the challenging one-page scenario.

### D. Ablation on WF-A2D

To comprehensively evaluate the effectiveness of WF-A2D, we have conducted two ablation experiments. The first experiment examines the individual contribution of each stage, while the second investigates different strategies for combining packet direction and length optimizations.

*1) Independent Component Analysis:* As shown in the Figure 5, the first ablation experiment individually assesses

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

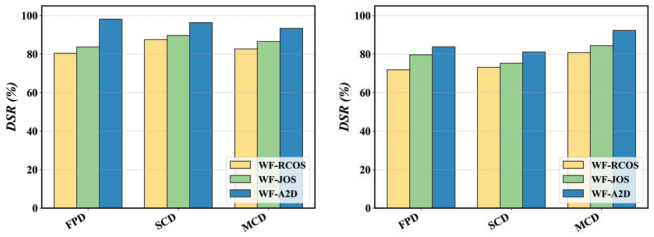



Fig. 6. Defense performance of different adversarial learning strategies using (a) NeuTic; (b) TAM-CNN.

the defense performance of adversarial learning approaches targeting packet direction and packet length patterns. When employing adversarial learning for packet direction pattern (*w/o len.*), the defense effectiveness against classifiers including DeepCorr, AppSniffer, and Var-CNN, has proved comparable to the defense performance provided by WF-A2D. This is also evident in the SCD dataset, where this approach successfully deceives FS-Net, performing comparably to WF-A2D. However, in other datasets, this approach did not match WF-A2D's overall ability to evade FS-Net and NeuTic. Despite these variations, the initial success of *w/o len.* demonstrates its effectiveness in masking discernible traffic patterns. Regarding the adversarial learning for packet length pattern (*w/o dir.*), this approach alone did not achieve the same defense performance as WF-A2D across the three datasets. Nonetheless, it demonstrates protective capabilities comparable to *w/o len.* in specific scenarios, such as against FS-Net in the FPD dataset, NeuTic in the SCD dataset, and TAM-CNN in the MCD dataset. These results validate its effectiveness and confirm that *w/o dir.* can successfully select a reasonable perturbation position range.

Overall, although each adversarial learning approach demonstrates effectiveness, none are sufficient on their own to provide a robust WF defense against various traffic analysis tools. Therefore, integrating both the packet direction and packet length learning approaches through cascading learning is essential and effective. WF-A2D ensures a more formidable defense across a broader spectrum of traffic analysis tools.

*2) Strategy Integration Analysis:* To evaluate the effectiveness of WF-A2D's two-stage cascading adversarial learning strategy, we develop two alternative approaches: WF-RCOS (Reversed Cascading Optimization Strategy), which prioritizes packet length obfuscation before directional modifications, and WF-JOS (Joint Optimization Strategy), which iteratively alternates between directional and length-based perturbations until convergence. Both strategies are assessed using two state-of-the-art traffic classifiers, NeuTic and TAM-CNN, to comprehensively analyze their defense performance

As illustrated in Figure 6, the experimental results highlight the superiority of WF-A2D's cascading approach. WF-RCOS exhibits weaker defense performance due to premature convergence, modifying packet lengths too early restricts the effectiveness of subsequent packet insertions, leaving directional features insufficiently obfuscated. While WF-JOS performs better than WF-RCOS by facilitating iterative

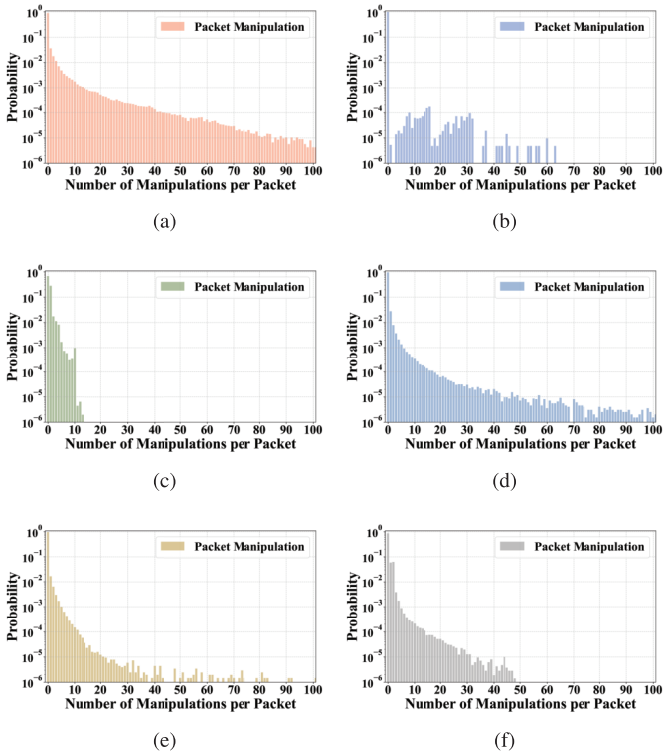



Fig. 7. Statistical distribution of the number of manipulations per packet: (a) Mockingbird; (b) Minipatch; (c) Prism; (d) FRONT; (e) WTF-PAD; (f) WF-A2D.

knowledge exchange between components, the simultaneous adjustment of both feature sets introduces conflicts, ultimately reducing the overall effectiveness of perturbations and leading to suboptimal defense performance.

These findings reinforce the advantages of WF-A2D's two-stage design. By first establishing robust directional disguise through packet insertion, followed by length-based refinements at a finer granularity, WF-A2D achieves comprehensive traffic obfuscation. This sequential optimization prevents premature convergence and ensures that each stage contributes maximally to overall defense robustness.

## E. Efficiency Analysis

In this subsection, we quantitatively assess the potential latency introduced by each WF defense scheme by measuring the number of manipulations applied to each packet before transmission. To achieve the desired defense performance, some packets undergo multiple manipulations, including padding, splitting, and insertion, as specified by each WF defense scheme. Since packets are sent sequentially, a higher probability of extensive manipulations on a single packet is prone to causing congestion and delays for subsequent packets, ultimately impacting user experience. For our analysis, we treat packet padding and packet insertion as a single operation, while packet splitting is counted as two operations due to the need to send two packets separately. We then illustrate the distribution of manipulation frequencies, as shown in Figure 7.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

Mockingbird clearly introduces the most packet manipulations, with the highest probability across the 1-100 range. This indicates that Mockingbird frequently alters packets, which can significantly affect latency and degrade user experience. Minipatch, which generates perturbations by sending dummy packet blocks, shows a discontinuous distribution of packet manipulations. By inserting fewer dummy packet blocks, Minipatch achieves lower bandwidth consumption; however, this results in weaker defense performance due to insufficient packet alterations.

Prism, on the other hand, enhances defense performance by manipulating a large number of packets, with an almost equal distribution of manipulated and unmanipulated packets. Additionally, it limits the number of manipulations per packet to between 1 and 15. However, as a real-time decision-making method, Prism waits for packets to arrive before determining whether to pad or split them based on frequency transition information. This process unintentionally introduces additional latency and increases congestion due to the higher number of calculation queries required.

In contrast, WF-A2D, WTF-PAD, and FRONT are pre-generated defense schemes that do not require real-time decision-making for obfuscation, thereby enhancing perturbation efficiency. WF-A2D, like the other two low-latency schemes, prefers to apply packet manipulations within the range of 1 to 5, with a probability less than 0.1. However, WTF-PAD and FRONT introduce excessive packet manipulations, with some packets undergoing more than 50 operations. This excessive manipulation increases latency compared to WF-A2D, making WF-A2D a more suitable choice when latency is considered.

### F. Retraining Resistance

To further assess the robustness of WF-A2D, we evaluate its performance against an adaptive adversary capable of retraining traffic analysis classifiers using obfuscated traffic samples. In this context, the adversary is aware of the WF defense mechanisms and can incorporate obfuscated traffic into adversarial training, enhancing classifiers' ability to recognize such flows. However, since the end-to-end privacy protection scheme and packet manipulations are concealed by QUIC encryption encapsulation, the adversary cannot determine whether specific traffic has undergone defense implementation. As a result, the adversary's classifiers must be capable of accurately identifying both obfuscated and undefended traffic flows, ensuring effective classification across diverse traffic patterns. For a comprehensive evaluation, two distinct scenarios are evaluated based on the adversary's knowledge of the WF defense schemes: a partially knowledgeable adaptive and a fully knowledgeable adaptive.

*1) Partially Knowledgeable Adaptive Adversary:* In this scenario, the adversary is aware of the WF-A2D but lacks knowledge of the specific discriminator used in generating the perturbation vectors. To address this limitation, the adversary employs advanced traffic analysis tools as surrogate discriminators to approximate the behavior of the defense. Specifically, TAM-CNN and NeuTic are selected due to their strong performance in analyzing directional packet patterns and packet length distributions. Using these surrogates, the adversary generates perturbation vectors and retrains classifiers with both obfuscated and undefended traffic samples.

TABLE IV

*F*1 *score* OF THE SIX WF DEFENSE SCHEMES UNDER FULL-KNOWLEDGE ADVERSARIAL TRAINING (MEAN±STD). BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST PERFORMANCE, RESPECTIVELY

| | NeuTic | | TAM-CNN | |
|---|---|---|---|---|
| | *w/o AT* | *w AT* | *w/o AT* | *w AT* |
| FRONT [32] | 55.60±0.1842 | 66.16±0.1667 | <u>55.76</u>±0.1431 | 73.98±0.1405 |
| WTF-PAD [31] | <u>52.43</u>±0.1908 | <u>55.04</u>±0.2466 | 59.03±0.1604 | 74.55±0.1039 |
| Mockingbird [39] | 54.40±0.1796 | 74.58±0.1541 | 60.16±0.1557 | 67.51±0.1829 |
| Minipatch [40] | 74.28±0.2357 | 85.25±0.1727 | 69.77±0.1953 | 90.01±0.0745 |
| Prism [12] | 53.68±0.2152 | 62.94±0.2279 | 60.07±0.1805 | <u>66.40</u>±0.2091 |
| WF-A2D | **49.97**±0.1887 | **50.91**±0.2438 | **53.41**±0.1436 | **57.27**±0.2810 |

For evaluation, the *DSR* is used to measure how well the classifiers learn to identify obfuscated traffic, and the *F*1 *score* assesses the overall identification performance in classifying both obfuscated and undefended traffic flows. The experiments are conducted on the SCD dataset, which presents a challenging case due to WF-A2D's lower average *DSR* and reduced *DG*. The results reveal that without adversarial training (*w/o AT*), WF-A2D achieves a *DSR* of 87.40% against TAM-CNN and 95.90% against NeuTic, with corresponding *F*1 *score* of 53.41% and 50.25%, respectively. After the adversary applies adversarial training (*w AT*), the *DSR* of WF-A2D decreases to 68.60% for TAM-CNN and 91.60% for NeuTic, indicating some decline in obfuscation effectiveness. However, this improvement in the adversary's ability to identify obfuscated traffic comes at a significant cost: their classifiers' *F*1 *score* drop to 45.38% for TAM-CNN and 29.52% for NeuTic. These results demonstrate that surrogate-based adversarial training fails to compromise WF-A2D's defense mechanism, as the adversary cannot maintain adequate classification performance across both obfuscated and undefended traffic.

*2) Fully Knowledgeable Adaptive Adversary:* In this scenario, the adversary possesses complete knowledge of WF-A2D, enabling them to replicate its perturbation mechanisms. To provide a comprehensive evaluation, we expand the scope to include five other state-of-the-art WF defense schemes, assuming the adversary also has complete knowledge of these defenses.

The experimental results in Table IV reveal that before adversarial training (*w/o AT*), WF-A2D achieves superior performance with *F*1 *score* of 49.97% and 53.41% against NeuTic and TAM-CNN, respectively. After adversarial training (*w AT*), WF-A2D maintains its robustness with only modest *F*1 *score* increases to 50.91% for NeuTic and 57.27% for TAM-CNN. In contrast, other defenses show substantial vulnerability to adversarial training, with Minipatch's *F*1 *score* reaching 85.25% for NeuTic and 90.01% for TAM-CNN, and Prism experiencing nearly 10% increase under NeuTic. These results demonstrate that other defenses retain exploitable traffic patterns, whereas WF-A2D's comprehensive obfuscation approach significantly limits the adversary's ability to learn and classify both undefended and obfuscated traffic effectively, even after adversarial retraining.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

### G. Discussion

Experimental results across three datasets demonstrate that WF-A2D achieves an average defense success rate of 96.38% against seven advanced WF schemes, representing its strong defensive capabilities. However, when deploying WF-A2D in real-world environments, two scenarios need to be considered.

Although WF-A2D currently focuses on client-to-server traffic manipulation, adversaries may attempt to circumvent this by targeting server-to-client flows. While dynamic content patterns and QUIC's packet scheduling behavior inherently complicate downstream traffic analysis, static websites with fixed content structures may still exhibit identifiable server-to-client patterns. To address this potential vulnerability, WF-A2D could be extended with client-side strategies that indirectly perturb server-to-client traffic, such as ACK manipulation or redundant request injection. These strategies, while promising, require further investigation to balance obfuscation effectiveness against bandwidth and latency costs.

As websites evolve over time, significant changes in their structure or behavior may alter upstream traffic characteristics. Although WF-A2D is designed to be resilient to minor variations through its focus on client-to-server traffic and position-based perturbation encoding, periodic updates to perturbation vectors are necessary to maintain optimal defense effectiveness. We propose two complementary strategies to address this challenge: (i) dynamic regeneration of perturbation vectors based on observed traffic drift, and (ii) hybrid updates that integrate historical and real-time traffic data. These mechanisms enable WF-A2D to remain adaptive and effective in the face of evolving web content and traffic analysis techniques.

## VI. Conclusion

In this study, we introduce WF-A2D, a novel asymmetric adversarial defense scheme for HTTP/3 website browsing. Our approach employs a two-stage cascading adversarial learning strategy to pre-generate position-based perturbation vectors, enabling packet-by-packet operations on the client side. The inherent features of QUIC support these asymmetric manipulations. Our framework demonstrates effective resistance to more powerful website traffic analysis tools. Furthermore, WF-A2D offers a way to support end-to-end privacy protection in a proxy-free manner, providing a practical and efficient solution for WF defense schemes.

Although the proposed WF defense method effectively enhances the ease of deployment, defense performance, and reduces bandwidth overhead, advancements in traffic analysis techniques, such as multi-sample temporal correlation analysis and multi-scale traffic analysis, continue to improve WF attacks. Therefore, future work should consider incorporating adversarial learning for a broader range of traffic scales and ultra-long traffic feature sequences to further enhance resistance against more complex WF attacks.

## References


[1] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2022.

[2] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[3] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, p. 23477.

[4] P. Liu, L. He, and Z. Li, "A survey on deep learning for website fingerprinting attacks and defenses," *IEEE Access*, vol. 11, pp. 26033–26047, 2023.

[5] W. De la Cadena et al., "TrafficSliver: Fighting website fingerprinting attacks with traffic splitting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2020, pp. 1971–1985.

[6] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. 10th Annu. ACM Workshop Privacy Electron. Soc.*, Oct. 2011, pp. 103–114.

[7] X. Luo, P. Zhou, E. W. W. Chan, W. Lee, R. K. C. Chang, and R. Perdisci, "HTTPOS: Sealing information leaks with browser-side obfuscation of encrypted flows," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Jan. 2011, pp. 1–20.

[8] G. Cherubin, J. Hayes, and M. Juarez, "Website fingerprinting defenses at the application layer," *Proc. Privacy Enhancing Technol.*, vol. 2017, no. 2, pp. 186–203, Apr. 2017.

[9] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2705–2722.

[10] S. Shan, A. N. Bhagoji, H. Zheng, and B. Y. Zhao, "Patch-based defenses against web fingerprinting attacks," in *Proc. 14th ACM Workshop Artif. Intell. Secur.*, Nov. 2021, pp. 97–109.

[11] X. Xiao et al., "A comprehensive analysis of website fingerprinting defenses on tor," *Comput. Secur.*, vol. 136, Jan. 2024, Art. no. 103577.

[12] W. Li et al., "Prism: Real-time privacy protection against temporal network traffic analyzers," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2524–2537, 2023.

[13] R. Xie et al., "Cactus: Obfuscating bidirectional encrypted TCP traffic at client side," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7659–7673, 2024.

[14] M. Nasr, A. Bahramali, and A. Houmansadr, "DeepCorr: Strong flow correlation attacks on Tor using deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1962–1976.

[15] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. 23rd USENIX Secur. Symp. (USENIX Secur.)*, 2014, pp. 143–157.

[16] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. 25th USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2016, pp. 1187–1203.

[17] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.

[18] S. Oh, M. Lee, H. Lee, E. Bertino, and H. Kim, "AppSniffer: Towards robust mobile app fingerprinting against VPN," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2318–2328.

[19] K. Abe and S. Goto, "Fingerprinting attack on tor anonymity using deep learning," in *Proc. Asia–Pacific Adv. Netw. (APAN)*, vol. 42, Apr. 2016, pp. 15–20.

[20] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," 2017, *arXiv:1708.06376*.

[21] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2018, pp. 1928–1943.

[22] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient website fingerprinting attack based on deep learning," in *Proc. Privacy Enhancing Technol.*, vol. 2019, Jul. 2019, pp. 292–310.

[23] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, "Tik-Tok: The utility of packet timing in website fingerprinting attacks," in *Proc. Privacy Enhancing Technol.*, 2020, pp. 5–24.

[24] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jun. 2019, pp. 1171–1179.

[25] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.

[26] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.


Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.

[27] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.

[28] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.

[29] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. Secur. Privacy*, May 2012, pp. 332–346.

[30] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 227–238.

[31] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. 21st Eur. Symp. Res. Comput. Secur. (ESORICS)*, Sep. 2016, pp. 27–46.

[32] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 717–734.

[33] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. 26th USENIX Secur. Symp. (SEC)*, Aug. 2017, pp. 1375–1390.

[34] R. Nithyanand, X. Cai, and R. Johnson, "Glove: A bespoke website fingerprinting defense," in *Proc. 13th Workshop Privacy Electron. Soc.*, Nov. 2014, pp. 131–134.

[35] M. Rigaki and S. Garcia, "Bringing a GAN to a knife-fight: Adapting malware communication to avoid detection," in *Proc. IEEE Security Privacy Workshops (SPW)*, Mar. 2018, pp. 70–75.

[36] J. Li, L. Zhou, H. Li, L. Yan, and H. Zhu, "Dynamic traffic feature camouflaging via generative adversarial networks," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Jun. 2019, pp. 268–276.

[37] J. Gong, W. Zhang, C. Zhang, and T. Wang, "Surakav: Generating realistic traces for a strong website fingerprinting defense," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 1558–1573.

[38] H. Liu, A. F. Diallo, and P. Patras, "Amoeba: Circumventing ML-supported network censorship via adversarial reinforcement learning," *Proc. ACM Netw.*, vol. 1, no. 3, pp. 1–25, Nov. 2023.

[39] M. S. Rahman, M. Imani, N. Mathews, and M. Wright, "Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1594–1609, 2021.

[40] D. Li, Y. Zhu, M. Chen, and J. Wang, "Minipatch: Undermining DNN-based website fingerprinting with adversarial patches," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2437–2451, 2022.

[41] M. A. I. M. Aminuddin, Z. F. Zaaba, A. Samsudin, F. Zaki, and N. B. Anuar, "The rise of website fingerprinting on tor: Analysis on techniques and assumptions," *J. Netw. Comput. Appl.*, vol. 212, Mar. 2023, Art. no. 103582.

[42] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.

[43] B. Gao, W. Liu, G. Liu, and F. Nie, "Resource knowledge-driven heterogeneous graph learning for website fingerprinting," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 3, pp. 968–981, Jun. 2024.

[44] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, "SnWF: Website fingerprinting attack by ensembling the snapshot of deep learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1214–1226, 2022.

[45] P. Zhan, L. Wang, and Y. Tang, "Website fingerprinting on early QUIC traffic," *Comput. Netw.*, vol. 200, Dec. 2021, Art. no. 108538.

[46] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.

[47] Z. Gu et al., "IEPSBP: A cost-efficient image encryption algorithm based on parallel chaotic system for green IoT," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 89–106, Mar. 2022.

[48] J.-P. Smith, L. Dolfi, P. Mittal, and A. Perrig, "QCSD: A QUIC client-side website-fingerprinting defence framework," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 771–789.

[49] T. Wang, "The one-page setting: A higher standard for evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2794–2806.

[50] N. Erickson et al., "AutoGluon-tabular: Robust and accurate AutoML for structured data," 2020, *arXiv:2003.06505*.




**Jianan Huang** (Student Member, IEEE) received the B.E. degree in electronic and information engineering from Zhejiang Agriculture and Forestry University in 2021. He is currently pursuing the Ph.D. degree with Nanjing University of Science and Technology. His research interests include network traffic obfuscation, network covert channels, and network traffic analysis.



**Weiwei Liu** (Member, IEEE) received the B.S. degree in automation and the Ph.D. degree in control science and engineering from Nanjing University of Science and Technology in 2010 and 2015, respectively. From 2014 to 2015, he was a Visiting Scholar with the Department of Computer Science, University of California at Davis. He is currently an Associate Professor with Nanjing University of Science and Technology. His research interests include network covert communication and encrypted traffic analysis.



**Guangjie Liu** (Member, IEEE) received the B.S. degree in electrical and computer engineering and the Ph.D. degree in control science and engineering from Nanjing University of Science and Technology in 2002 and 2007, respectively. From 2016 to 2017, he was a Visiting Scholar with the Department of Computer Science, University of California at Davis. He is currently a Professor with Nanjing University of Information Science and Technology. His research interests include network and information security.



**Bo Gao** (Member, IEEE) received the B.S. degree in physics from Jilin University, Jilin, in 2016, and the M.Eng. degree in control engineering from Nanjing University of Science and Technology, Nanjing, China, in 2020, where he is currently pursuing the Ph.D. degree. His research interests include anonymity networks and network traffic analysis.



**Fengyuan Nie** received the B.S. degree in automation from Shanghai University of Electric Power in July 2020. He is currently pursuing the Ph.D. degree with Nanjing University of Science and Technology. His research interests include the Internet of Things, network traffic classification, and deep learning.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on August 13,2026 at 07:29:30 UTC from IEEE Xplore. Restrictions apply.